Two paths to a superhuman lap time

Two complementary controllers that share one learned dynamics model but split the navigation question differently.

① Min-Snap + RL (plan → track)

- **Collision-aware min-snap planner** lays a feasible path through the gates
- A PPO policy **tracks** it as look-ahead points
- Geometry = *where* to fly, RL = *how*; re-plans in ~1 ms as gates appear

② End-to-End Nav RL (sense → act)

- One PPO policy maps **raw race state** straight to **commands**
- **No explicit trajectory**, nothing to re-plan
- Fully reactive; removes the planner as a failure point

Why both? Hybrid = robust + interpretable (a visible reference path); end-to-end = maximally reactive, theoretically highest-performance, but more brittle and sensitive. A shared **debug UI** inspects, pauses, and compares either policy live for easier deployment.

Collision-aware geometric planner + learned path-tracker `train_min_snap_rl.py` / `min_snap_rl_controller.py`

Planning (classical, snap-optimal)

- **Min-snap** polynomial spline through gate centers, waypoints forced **perpendicular** to each gate plane
- Pole + gate-frame **keep-out corridors**; collision-aware seeding + clearance repair
- **Acceleration-limited** time scaling (`A_LIMIT=6.0`, cruise `SPEED=0.6`) → dynamically feasible
- 1st plan: full L-BFGS snap minimization; **re-plan** on sensed gate motion (`>4 cm`): fast seed + repair, ~1 ms in-loop

Tracking policy (RL)

- **Obs (73-D):** drone state (13) + 10 look-ahead path deltas @ `0.1 s` (30) + 2-step history (26) + last action (4)
- **Action:** 4-D `[r, p, y, thrust]`, yaw zeroed, scaled to $\pm\frac{\pi}{2}$ and the drone's thrust bounds
- Learned dynamics → tracks tight turns far better than the onboard state controller
- **PPO:** `2048` envs, `5.5 M` steps, $\gamma = 0.94$, 2×64 tanh MLP

$$r_t = \exp(-2d_t) - 0.06\|\text{rpy}\| - 0.02a_{\text{thr}}^2 - 0.4(\Delta a_{\text{thr}})^2 - 1.0\left\|\Delta a_{\text{xy}}\right\|^2 \quad \text{(on crash: } r_t = -1\text{)}$$

d_t distance to the current look-ahead point → $\exp(\cdot)$ rewards **hugging the path** (=1 on it, decays away) · out-of-bounds **crash** replaces it with -1 · $\|\text{rpy}\|$ keeps the drone **level** · last three penalize **thrust energy** + **thrust/xy jerk** for smooth flight

End-to-End Navigation RL

One policy, raw geometry → command `train_nav_rl.py` / `nav_rl_common.py`

Observation (egocentric)

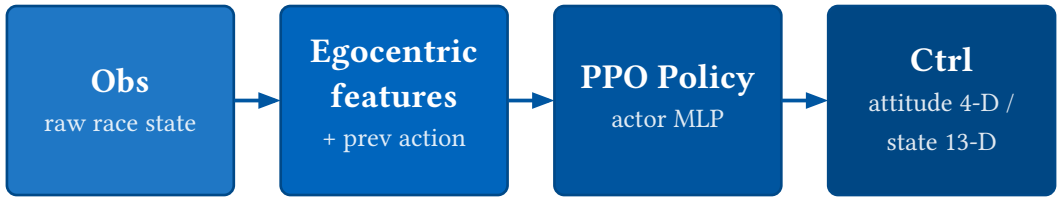
- Target-gate Δpos (world **and** body frame)
- Gate orientation: 9-D rotation matrix
- Linear + angular velocity
- 2 nearest obstacles, sensed-gate ratio
- Progress `target/N`, prev action `a_(t-1)`

Action & Reward

- Two control modes:
  - `attitude` — 4-D thrust + r/p/y
  - `state` — 13-D full setpoint
- **Shaped reward:** gate progress, gate-pass / success bonuses, crash + energy/jerk penalties:

$$r_t = r_{\text{base}} + 5(d_{t-1} - d_t) + 10 \cdot \mathbb{1}_{\text{pass}} + 15 \cdot \mathbb{1}_{\text{success}} - 5 \cdot \mathbb{1}_{\text{crash}} - 0.01 a_{\text{thr}}^2 - 0.25 \left\| \Delta a_{\text{rpy}} \right\|^2 - 0.1 (\Delta a_{\text{thr}})^2$$

r_{base} sparse env reward · d distance to target gate → reward **closing in** on it · $\mathbb{1}_{\text{pass}}$ bonus for crossing a gate · $\mathbb{1}_{\text{success}}$ bonus for finishing the track · $\mathbb{1}_{\text{crash}}$ penalty for crashing · last three terms penalize **thrust energy** and **jerk** (action change) for smooth flight



Standalone PPO · GAE · `2048` JAX envs · checkpoints store args + arch to rebuild the matching net

Live Debug UI

Stop, resume, and inspect the policy mid-flight `lsy_drone_racing/debug_ui/`

What it shows

- **Live 3D trajectory:** history + gates/obstacles
- Position & action time-series updated in real-time
- Sidebar: target gate, speed, action, rate of last step

Stop / start the policy

- Dashboard sends **stop / resume** over TCP to the `Controller`
- **Stop** → controller latches a **PID hover hold** at the current pose, safe to inspect
- **Resume** → control handed straight back to the policy

Figure 1: UI for debugging and human supervision of the drone policy on deployment